

Chapter 2: Hibernate Basics

- Creating and Executing Hibernate Application
- hibernate.cfg.xml
- Example 1
- Session Interface Methods
- Example 2

Creating and Executing Hibernate Application

To create a New Hibernate Project in Eclipse IDE using Maven:

1. File | New | Other | Maven | Maven Project
 - Click on Next
2. Select the Defaults
 - Click on Next
3. Select the Artifact as **maven-archetype-quickstart**
 - Click on Next
4. Specify the Group Id (package structure) and Artifact Id (project name)
 - Group Id : **com.vedisoft.hibernate**
 - Artifact Id : **test1**
 - Click on Finish

Adding the required Jar Files to Project

To add jar files in Maven Project, dependencies need to be added in **pom.xml**.

Add the following dependencies:

```
<!-- https://mvnrepository.com/artifact/org.hibernate/hibernate-core -->
<dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>5.4.17.Final</version>
</dependency>

<!-- https://mvnrepository.com/artifact/mysql/mysql-connector-java -->
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.20</version>
</dependency>
```

NOTE: After saving pom.xml Maven will download and add the mention jar files to your project. This may take some time.

Check the Project by executing the App.java file from the src/main/java folder

hibernate.cfg.xml

Hibernate requires to know in advance — where to find the mapping information that defines how your Java classes relate to the database tables. Hibernate also requires a set of configuration settings related to database and other related parameters. All such information is usually supplied as a standard Java properties file called **hibernate.properties**, or as an XML file named **hibernate.cfg.xml**.

Compiled by Rohit Ahuja

Hibernate Properties

Following is the list of important properties, you will be required to configure for a databases in a standalone situation –

1. hibernate.dialect: This property makes Hibernate generate the appropriate SQL for the chosen database.
2. hibernate.connection.driver_class: The JDBC driver class.
3. hibernate.connection.url: The JDBC URL to the database instance.
4. hibernate.connection.username: The database username.
5. hibernate.connection.password: The database password.
6. hibernate.connection.pool_size: Limits the number of connections waiting in the Hibernate database connection pool.
7. hibernate.connection.autocommit: Allows autocommit mode to be used for the JDBC connection.

To add hibernate.cfg.xml to an Eclipse Project

File | New | Other | Hibernate | Hibernate Configuration File (cfg.xml)

Sample: hibernate.cfg.xml

```
<hibernate-configuration>
    <session-factory>
        <property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property name="hibernate.connection.password">vedisoft</property>
        <property name="hibernate.connection.url">jdbc:mysql://localhost/hib</property>
        <property name="hibernate.connection.username">root</property>
        <property name="hibernate.dialect">org.hibernate.dialect.MySQL8Dialect</property>
        <property name="hbm2ddl.auto">update</property>
        <property name="show_sql">true</property>
        <mapping class="com.vedisoft.hibernate.test2.pojos.Employee" />
    </session-factory>
</hibernate-configuration>
```

Example 1: class1\test1

Pre-Requirement: Create a database by the **hib** in **MySQL 8**

1. test1\pom.xml
2. src\main\java\hibernate.cfg.xml
3. com.vedisoft.hibernate.test1
 - a. **HibernateUtil.java**

```
private static SessionFactory sessionFactory;

public static SessionFactory getSessionFactory() {
    if (sessionFactory == null) {
        StandardServiceRegistry standardRegistry = new
            StandardServiceRegistryBuilder().configure().build();
        Metadata metadata = new MetadataSources(standardRegistry).getMetadataBuilder().build();
        sessionFactory = metadata.getSessionFactoryBuilder().build();
    }
    return sessionFactory;
}
```

b. Employee.java

@Entity

```
public class Employee {
    @Id
    protected int empCode;
    protected String empName;
    protected String empDept;
    protected String empDesig;
    protected Date empDoj;
    protected int empBasic;
    // Constructors, Getter / Setters and Operations
}
```

c. App.java

```
Employee emp = new Employee(1, "Sanjay Jain", "Sales", "Manager", new java.util.Date(), 10000);
SessionFactory sessionFactory = HibernateUtil.getSessionFactory();
Session session = sessionFactory.openSession();
Transaction transaction = session.beginTransaction();
session.save(emp);
transaction.commit();
```

Session Interface Methods

1. **Transaction beginTransaction()** : Begin a unit of work and return the associated Transaction object.
2. **void cancelQuery()** : Cancel the execution of the current query.
3. **void clear()** : Completely clear the session.
4. **Connection close()** : End the session by releasing the JDBC connection and cleaning up.
5. **Criteria createCriteria(Class persistentClass)**: Create a new Criteria instance, for the given entity class, or a superclass of an entity class.
6. **Criteria createCriteria(String entityName)**: Create a new Criteria instance, for the given entity name.
7. **Serializable getIdentifier(Object object)**: Return the identifier value of the given entity as associated with this session.
8. **Query createFilter(Object collection, String queryString)**: Create a new instance of Query for the given collection and filter string.
9. **Query createQuery(String queryString)**: Create a new instance of Query for the given HQL query string.
10. **SQLQuery createSQLQuery(String queryString)**: Create a new instance of SQLQuery for the given SQL query string.
11. **void delete(Object object)**: Remove a persistent instance from the datastore.
12. **Session get(String entityName, Serializable id)**: Return the persistent instance of the given named entity with the given identifier, or null if there is no such persistent instance.
13. **SessionFactory getSessionFactory()**: Get the session factory which created this session.
14. **void refresh(Object object)**: Re-read the state of the given instance from the underlying database.
15. **Transaction getTransaction()**: Get the Transaction instance associated with this session.
16. **boolean isConnected()**: Check if the session is currently connected.
17. **boolean isDirty()**: Does this session contain any changes which must be synchronized with the database?
18. **boolean isOpen()**: Check if the session is still open.

19. **Serializable save(Object object)**: Persist the given transient instance, first assigning a generated identifier.
20. **void saveOrUpdate(Object object)**: Either save(Object) or update(Object) the given instance.
21. **void update(Object object)**: Update the persistent instance with the identifier of the given detached instance.

Example 2: class1 \ test2

Pre-Requisite: Create a database by the **hib** in **MySQL 8**

1. test2 \ **pom.xml**
2. src \ main \ java \ **hibernate.cfg.xml**
3. com.vedisoft.hibernate.test2.utils
 - **HibernateUtil.java**
4. com.vedisoft.hibernate.test2.pojos
 - **Employee.java** @Entity
 - **Name.java** @Embeddable
5. com.vedisoft.hibernate.test2.daos
 - **IEmployeeDao.java**
 - **EmployeeDao.java**
6. com.vedisoft.hibernate.test2
 - **App.java**